



BUAP

Benemérita Universidad de Puebla

Ingeniería en Ciencias de la Computación

Materia: Programación Distribuida y Aplicada

Alejandro Cholula Olvera- Fatima Mentado Girón

Manual Usuario- Scidist-archive

REPOSITORIO

github.com/alexvera23/Scidist-archive

ARQUITECTURA

Microservicios · P2P · gRPC · Docker

BACKEND

Node.js · Express · Mongoose · gRPC

FRONTEND

React 19 · Vite · Bootstrap 5 · Axios

IA

Python · FastAPI · HuggingFace mDeBERTa

BASE DE DATOS

MongoDB · Replica Set · Transacciones ACID

Fecha de entrega: 20-05-2026

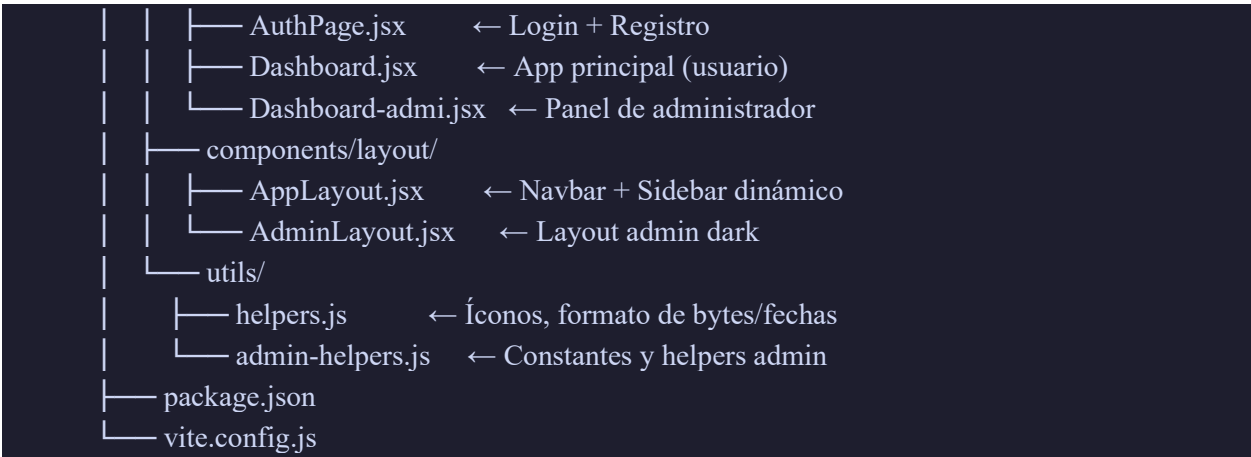


1. Estructura General del Proyecto

Scidist-Archive es un sistema distribuido para clasificar artículos científicos usando Inteligencia Artificial. Está compuesto por cuatro microservicios independientes más un frontend en React, todos orquestables con Docker.

1.1 Árbol de directorios completo

```
Scidist-archive/
├── proto/
│   └── storage.proto    ← Contrato gRPC compartido entre gateway y nodos
├── services/
│   ├── api-gateway/    ← Puerto 3000 — Entrada HTTP de toda la app
│   │   ├── server.js   ← Express + Multer + gRPC client + IA
│   │   ├── package.json
│   │   └── Dockerfile
│   ├── metadata-service/ ← Puerto 3001 (HTTP) + 3002 (UDP heartbeats)
│   │   ├── server.js   ← Express + Mongoose + Lógica de negocio
│   │   ├── models/index.js ← Todos los esquemas Mongoose
│   │   ├── package.json
│   │   └── Dockerfile
│   ├── storage-node/   ← Puerto 50051 (gRPC) — Almacenamiento físico
│   │   ├── server.js   ← gRPC server + replicación P2P + heartbeat UDP
│   │   ├── package.json
│   │   └── Dockerfile
│   └── classifier-service/ ← Puerto 8000 — Clasificación IA
│       ├── main.py      ← FastAPI + HuggingFace mDeBERTa
│       ├── requirements.txt
│       └── Dockerfile
└── Frontend/
    ├── scidist-frontend/ ← App React + Vite
    │   ├── src/
    │   │   ├── App.jsx
    │   │   ├── main.jsx
    │   │   ├── api/axiosConfig.js
    │   │   ├── routes/AppRouter.jsx
    │   │   ├── pages/
    │   │   └── Scidist.jsx ← Landing page
```



1.2 Tecnologías y Dependencias por Servicio

Servicio / App	Dependencias principales (package.json / requirements.txt)
api-gateway	express 5, multer 2, axios, @grpc/grpc-js, @grpc/proto-loader, pdfjs-dist 3, cors, crypto
metadata-service	express 5, mongoose 9, cors, dotenv
storage-node	@grpc/grpc-js, @grpc/proto-loader, axios, fs-extra
classifier-service	fastapi, uvicorn, transformers, torch, sentencepiece
scidist-frontend	react 19, react-dom, react-router-dom 7, axios, bootstrap 5, bootstrap-icons, react-bootstrap, vite 8

2. Cómo Ejecutar el Proyecto

El proyecto está diseñado para correr en Docker. Sin embargo, también puede ejecutarse en modo local para desarrollo. A continuación se detallan ambas formas.

2.1 Prerrequisitos

Herramienta	Versión mínima / Notas
Docker	24.0 o superior — para correr todos los servicios como contenedores
Docker Compose	v2.x — orquesta los contenedores con un solo comando
Node.js	18 LTS — para correr los servicios sin Docker en modo dev
Python	3.9+ — solo si corres el classifier-service sin Docker
npm	8+ — gestión de dependencias de los servicios Node y del frontend

2.2 Variables de entorno requeridas

El metadata-service requiere un archivo .env en su directorio con la URI de MongoDB:

```
# services/metadata-service/.env
MONGO_URI=mongodb://usuario:password@host:27017/scidist?replicaSet=rs0

# Si usas MongoDB Atlas:
MONGO_URI=mongodb+srv://usuario:password@cluster.mongodb.net/scidist
```

⚠ Si MONGO_URI no está definida, el metadata-service llama a process.exit(1) y el contenedor se detiene inmediatamente. Esta validación está en la línea de arranque del servidor.

2.3 Configurar la IP del Gateway en el Frontend

El frontend conecta al API Gateway a través de Tailscale VPN. La IP está hardcodeada en axiosConfig.js y debe cambiarse a la IP real de tu máquina:

```
// Frontend/scidist-frontend/src/api/axiosConfig.js

// Cambia esta IP a la IP de Tailscale de la máquina donde corre el Gateway:
const GATEWAY_URL = "http://100.119.151.81:3000/api/v1";

const api = axios.create({
  baseURL: GATEWAY_URL,
  headers: { 'Content-Type': 'application/json' }
```

```
});
```

```
export default api;
```

💡 **Red local** Si corres todo en la misma máquina sin Tailscale, cambia la URL a:
`http://localhost:3000/api/v1`

2.4 Instalar dependencias y correr en modo desarrollo (sin Docker)

Gateway (Node.js)

```
cd services/api-gateway
npm install
# Asegúrate de que proto/ está disponible
node server.js
# Escucha en: http://localhost:3000
```

Metadata Service (Node.js)

```
cd services/metadata-service
npm install
# Requiere el archivo .env con MONGO_URI
node server.js
# Escucha en: http://localhost:3001 (HTTP) y UDP 3002
```

Storage Node (Node.js)

```
cd services/storage-node
npm install
# Variables de entorno necesarias:
NODE_ID=storage-node-local PORT=50051 METADATA_HOST=localhost node server.js
# Escucha en: 0.0.0.0:50051 (gRPC)
```

Classifier Service (Python)

```
cd services/classifier-service
pip install -r requirements.txt
# Descarga el modelo mDeBERTa (~1.5 GB) la primera vez
uvicorn main:app --host 0.0.0.0 --port 8000
```

Frontend (React + Vite)

```
cd Frontend/scidist-frontend
npm install
npm run dev
# Abre en: http://localhost:5173
```

2.5 Flujo de Comunicación entre Servicios

USUARIO (Navegador)

| HTTP/REST



API GATEWAY :3000

- |
 - ├── POST /api/v1/upload
 - |
 - ├── 1. Valida extensión (.pdf obligatorio)
 - ├── 2. Extrae texto con pdfjs-dist (primeras 3 páginas)
 - ├── 3. HTTP → classifier-service:8000 (Validación IMRyD)
 - ├── 4. HTTP → classifier-service:8000 (Clasificación temática)
 - ├── 5. HTTP → metadata-service:3001 (Obtiene categorías del usuario)
 - ├── 6. HTTP → metadata-service:3001 (Obtiene nodos activos)
 - ├── 7. gRPC → storage-node:50051 (Transfiere el archivo binario)
 - └── 8. HTTP → metadata-service:3001 (Guarda metadatos + StorageMap)
 - ├── GET /api/v1/download/:hash
 - | HTTP → metadata-service:3001 (Localiza el nodo)
 - └── gRPC → storage-node:50051 (Streaming de bytes al cliente)
 - ├── DELETE /api/v1/delete/:hash
 - └── HTTP → metadata-service:3001 (Borrado lógico + tareas DELETE)
 - └── Puentes de Auth, Temas, Admin → metadata-service:3001

STORAGE NODES (background)

- |
 - ├── UDP → metadata-service:3002 (Heartbeat cada 5s)
 - ├── HTTP → metadata-service:3001 (Registro al arrancar)
 - └── gRPC → otros storage-nodes (Replicación P2P cada 15s)

3. proto/storage.proto — Contrato gRPC

El archivo proto define el contrato de comunicación binaria entre el API Gateway y los Storage Nodes. Es la única forma en que los nodos reciben y envían archivos.

```
// proto/storage.proto
syntax = "proto3";
package storage;

service StorageService {
    // El cliente envía chunks de bytes (streaming del cliente)
    rpc UploadFile (stream UploadRequest) returns (UploadResponse);

    // El cliente pide un hash y recibe un flujo de bytes (streaming del servidor)
    rpc DownloadFile (DownloadRequest) returns (stream DownloadResponse);
}

// UploadRequest usa 'oneof' para distinguir entre
// el primer mensaje (metadatos) y el resto (datos binarios)
message UploadRequest {
    oneof data {
        FileInfo info = 1; // Primer chunk: hash y extensión
        bytes chunk = 2; // Chunks siguientes: datos del archivo
    }
}

message FileInfo {
    string file_hash = 1; // SHA-256 del archivo completo
    string extension = 2; // Ej: .pdf
}

message UploadResponse {
    string file_hash = 1;
    bool success = 2;
    string message = 3;
}

message DownloadRequest {
    string file_hash = 1;
}
```

```
message DownloadResponse {  
  bytes chunk = 1; // 64 KB por chunk (highWaterMark del readStream)  
}
```

El proto se carga dinámicamente en tiempo de ejecución tanto en el Gateway (cliente gRPC) como en cada Storage Node (servidor gRPC):

```
// Carga del proto en api-gateway/server.js y storage-node/server.js  
const PROTO_PATH = path.join(__dirname, 'proto', 'storage.proto');  
  
const packageDefinition = protoLoader.loadSync(PROTO_PATH, {  
  keepCase: true, // Respeta mayúsculas/minúsculas de los campos  
  longs: String,  
  enums: String,  
  defaults: true,  
  oneofs: true // Necesario para el campo 'oneof data' de UploadRequest  
});  
  
const storageProto = grpc.loadPackageDefinition(packageDefinition).storage;  
// storageProto.StorageService ahora es el cliente/servidor gRPC
```

4. API Gateway — services/api-gateway/server.js

Es la única puerta de entrada de la aplicación. Los clientes (el frontend React y cualquier herramienta HTTP) solo hablan con este servicio en el puerto 3000. Internamente reenvía las peticiones a los demás microservicios.

4.1 Configuración Inicial

```
// Middlewares globales
app.use(express.json());
app.use(cors()); // Permite peticiones desde cualquier origen (React en :5173)

// Multer: almacenamiento temporal en disco antes de enviarlo por gRPC
const upload = multer({ dest: 'temp_uploads/' });
// Los archivos se guardan como temp_uploads/<hash-aleatorio> sin extensión
// y se eliminan siempre con fs.unlinkSync() al terminar el procesamiento
```

4.2 Lógica de Anillo (Ring Hashing) para Replicación

Cada vez que se sube un archivo, el gateway calcula en qué nodo almacenarlo y cuáles serán las dos réplicas, usando el SHA-256 del archivo como clave de distribución:

```
// api-gateway/server.js — getRoutingNodes()
function getRoutingNodes(fileHash, activeNodes) {
  if (activeNodes.length === 0) return null;

  // Convierte los primeros 8 caracteres del hash a entero
  const hashInt = parseInt(fileHash.substring(0, 8), 16);

  // Nodo primario: posición en el anillo de nodos activos
  const primaryIndex = hashInt % activeNodes.length;

  // Réplicas: los dos nodos siguientes en el anillo (circular)
  const replica1Index = (primaryIndex + 1) % activeNodes.length;
  const replica2Index = (primaryIndex + 2) % activeNodes.length;

  return {
    primary: activeNodes[primaryIndex],
    replicas: [activeNodes[replica1Index], activeNodes[replica2Index]]
  };
}
```

4.3 POST /api/v1/upload — Flujo completo

Este es el endpoint más complejo del sistema. Ejecuta 8 pasos secuenciales:

```
// Paso 1 — Validación de extensión (solo PDF permitido)
if (extension !== '.pdf') {
  fs.unlinkSync(tempPath);
  return res.status(400).json({ error: 'Solo se aceptan archivos PDF.' });
}

// Paso 2 — Cálculo del hash SHA-256 del archivo completo
const hashSum = crypto.createHash('sha256');
hashSum.update(fileBuffer);
const fileHash = hashSum.digest('hex');

// Paso 3 — Extracción de texto con pdfjs-dist (primeras 3 páginas)
const numPages = Math.min(3, pdfDocument.numPages);
// Se limita a 2500 caracteres para no saturar el modelo de IA
extractedText = fullText.replace(/\s+/g, ' ').substring(0, 2500);

// Paso 4 — Obtener categorías del usuario (GET al metadata-service)
const catResponse = await axios.get(
  `http://metadata-service:3001/api/v1/users/${userId}/categories`
);

// Paso 5A — Validación IMRyD con el classifier-service
// Etiquetas: ['artículo científico (IMRyD)', 'documento genérico', 'publicidad o spam']
// Si confidence < 0.4 → se rechaza el archivo con HTTP 400

// Paso 5B — Clasificación temática (si pasó la validación IMRyD)
// Usa las categorías del usuario como candidateLabels
// Si confidence > 0.3 → asigna el tema ganador

// Paso 6 — Consulta nodos activos al metadata-service
const activeNodes = await fetchActiveNodes();
// fetchActiveNodes() → GET http://metadata-service:3001/api/v1/nodes

// Paso 7 — Transferencia gRPC con failover automático
// Intenta nodo primario → réplica 1 → réplica 2
// Si los 3 fallan → HTTP 503
for (const node of [primary, ...replicas]) {
  // gRPC streaming: primero envía FileInfo, luego el buffer completo
  call.write({ info: { file_hash: fileHash, extension: '.pdf' } });
}
```

```

call.write({ chunk: fileBuffer });
call.end();
}

// Paso 8 — Guardar metadatos en MongoDB (POST al metadata-service)
await axios.post('http://metadata-service:3001/api/v1/articles', {
  file_hash, title: originalName,
  owner_id: userId,
  theme_id: finalThemeId,
  subtheme_id: finalSubthemeId,
  node_id: successfulNode.node_id,
  replicas: [ids de los otros nodos],
  size: fileBuffer.length
});

```

4.4 GET /api/v1/download/:hash

```

// 1. Verifica header x-user-id (obligatorio)
// 2. Consulta nodos activos
// 3. GET al metadata-service para saber en qué nodo está el archivo
//    → /api/v1/articles/:hash?owner_id=userId
// 4. Conecta gRPC al nodo encontrado y hace streaming de los chunks
//    al cliente HTTP (response.write por cada chunk de 64 KB)

res.setHeader('Content-Disposition', `attachment; filename="${title}"`);
res.setHeader('Content-Type', 'application/octet-stream');

call.on('data', (response) => res.write(response.chunk));
call.on('end', () => res.end());
call.on('error', (err) => { if (!res.headersSent) res.status(500).json({...}) });

```

4.5 DELETE /api/v1/delete/:hash

El borrado es lógico primero (desasigna el archivo del usuario) y físico después si nadie más lo tiene:

```

// El gateway simplemente reenvía al metadata-service:
await axios.delete(
  `http://metadata-service:3001/api/v1/articles/${fileHash}?owner_id=${userId}`
);
// El metadata-service implementa la lógica completa de reference counting

```

4.6 Middleware de Seguridad Admin — checkAdmin

```
// api-gateway/server.js — checkAdmin middleware
const checkAdmin = async (req, res, next) => {
  const userId = req.headers['x-user-id'];
  if (!userId) return res.status(401).json({ error: 'No autorizado' });

  // Consulta al metadata-service si el usuario tiene is_admin: true
  const { data } = await axios.get(
    `http://metadata-service:3001/api/v1/admin/check/${userId}`
  );

  if (!data.is_admin) return res.status(403).json({ error: 'Acceso denegado' });
  next(); // El usuario SÍ es admin, continúa al endpoint
};

// Se aplica a todos los endpoints /api/v1/admin/*:
app.get('/api/v1/admin/users', checkAdmin, async (req, res) => {...});
app.delete('/api/v1/admin/users/:id', checkAdmin, async (req, res) => {...});
app.get('/api/v1/admin/nodes', checkAdmin, async (req, res) => {...});
app.get('/api/v1/admin/articles', checkAdmin, async (req, res) => {...});
app.get('/api/v1/admin/replications', checkAdmin, async (req, res) => {...});
```

4.7 Tabla completa de Endpoints del Gateway

Método + Ruta	Descripción y destino interno
POST /api/v1/upload	Sube PDF: valida, extrae texto, llama IA, distribuye por gRPC, guarda metadatos
GET /api/v1/download/:hash	Descarga el archivo del nodo que lo contiene, streaming gRPC → HTTP
DELETE /api/v1/delete/:hash	Borrado lógico + dispara tareas DELETE en nodos si eres el último dueño
POST /api/v1/auth/register	Puente → metadata-service: crea usuario + categoría General por defecto
POST /api/v1/auth/login	Puente → metadata-service: valida email/password, devuelve datos del usuario
GET /api/v1/categories/available	Puente → metadata-service: devuelve el CATALOGO_CATEGORIAS estático
GET /api/v1/categories/catalog	Puente → metadata-service: catálogo maestro global
GET /api/v1/themes/user/:id	Puente → metadata-service: árbol de categorías del usuario (para sidebar)

GET /api/v1/files/user/:id	Puente → metadata-service: artículos del usuario con populate de temas
GET /api/v1/themes/me	Categorías del usuario logueado (usa header x-user-id)
POST /api/v1/themes/me	Agrega nuevas temáticas al usuario (usa preferences: { Redes: ['TCP'] })
DELETE /api/v1/themes/me/:id	Elimina tema; reasigna artículos a General/Otros
DELETE /api/v1/subthemes/me/:id	Elimina subtema; reasigna archivos a 'Otros' del mismo tema padre
GET /api/v1/admin/users	Admin: lista todos los usuarios (checkAdmin)
POST /api/v1/admin/users	Admin: crea usuario (checkAdmin)
PUT /api/v1/admin/users/:id	Admin: edita usuario (checkAdmin)
DELETE /api/v1/admin/users/:id	Admin: elimina usuario en cascada (checkAdmin)
GET /api/v1/admin/nodes	Admin: estado de salud de todos los nodos (checkAdmin)
GET /api/v1/admin/articles	Admin: inventario global de artículos (checkAdmin)
GET /api/v1/admin/replications	Admin: últimas 100 tareas de replicación (checkAdmin)
GET /api/v1/admin/storage-maps	Admin: mapa de almacenamiento por archivo y nodo (checkAdmin)
GET /api/v1/admin/users/:id/themes	Admin: árbol de categorías con IDs de un usuario específico (checkAdmin)
POST /api/v1/admin/users/:id/themes	Admin: añade categorías a un usuario existente (checkAdmin)
DELETE /api/v1/admin/themes/:id	Admin: elimina tema completo con reasignación de artículos (checkAdmin)
DELETE /api/v1/admin/subthemes/:id	Admin: elimina subtema con reasignación de artículos (checkAdmin)

5. Metadata Service — `services/metadata-service/`

Es el cerebro del sistema: gestiona todos los datos de usuarios, categorías, artículos, nodos y tareas de replicación. Corre en Express + Mongoose sobre MongoDB con soporte para Replica Set y transacciones ACID.

5.1 Modelos Mongoose — `models/index.js`

Ocho colecciones definen el esquema completo de datos:

Modelo	Campos principales y propósito
User	username, email, password, is_admin (default: false) — timestamps: true
Theme	name, owner_id (ref: User) — Nivel 1 de clasificación por usuario
Subtheme	name, parent_theme_id (ref: Theme), owner_id — Nivel 2 de clasificación
Article	file_hash (indexed), title, owner_id, theme_id, subtheme_id, size, status: ['uploading','available','deleted','error'] — timestamps: true
StorageMap	file_hash (indexed), node_id, is_primary, version, status: ['synced','error']
ReplicationTask	file_hash, source_node, target_node, task_type: ['REPLICATE','DELETE'], status: ['pending','in_progress','done','failed'], retry_count — timestamps: true
NodeHealth	node_id (indexed), status: ['up','down'], last_heartbeat — actualizado por UDP
ActiveNode	node_id (unique), address (ej: '100.x.x.x:50051'), last_seen — directorio de nodos

5.2 Servidor UDP — Heartbeats de los Nodos

Los Storage Nodes envían un mensaje UDP cada 5 segundos. El metadata-service tiene un servidor UDP en el puerto 3002 que los recibe y actualiza NodeHealth:

```
// metadata-service/server.js — Servidor UDP
const udpServer = dgram.createSocket('udp4');

udpServer.on('message', async (msg, rinfo) => {
  const data = JSON.parse(msg); // { node_id, status, timestamp }
  if (data.node_id) {
    await NodeHealth.findOneAndUpdate(
      { node_id: data.node_id },
```

```

    { status: 'up', last_heartbeat: new Date(data.timestamp) },
    { upsert: true, new: true } // Crea si no existe
  );
}
});

udpServer.bind(3002, '0.0.0.0');

// Cada 10 segundos marca como 'down' los nodos sin heartbeat en 15s
setInterval(async () => {
  const threshold = new Date(Date.now() - 15000);
  await NodeHealth.updateMany(
    { last_heartbeat: { $lt: threshold }, status: 'up' },
    { $set: { status: 'down' } }
  );
}, 10000);

```

5.3 Registro de usuarios con preferencias

Al registrarse, el usuario elige sus áreas de interés. El backend crea automáticamente la carpeta General y todas las carpetas elegidas con sus subcategorías:

```

// metadata-service/server.js — POST /api/v1/auth/register
// Body esperado:
// { username, email, password, preferences: { 'Redes': ['Protocolos'], 'Linux': [] } }

// 1. Crea el usuario
const newUser = new User({ username, email, password });
await newUser.save();

// 2. Siempre crea General + General/Otros
const generalTheme = new Theme({ name: 'General', owner_id: newUser._id });
await generalTheme.save();
await new Subtheme({ name: 'Otros', parent_theme_id: generalTheme._id, ... }).save();

// 3. Itera preferences para crear temas y subtemas elegidos
for (const [themeName, subthemes] of Object.entries(preferences)) {
  if (themeName.toLowerCase() === 'general') continue; // Ya fue creado
  const theme = new Theme({ name: themeName, owner_id: newUser._id });
  await theme.save();
  // Subcategoría 'Otros' por defecto en cada tema
  await new Subtheme({ name: 'Otros', parent_theme_id: theme._id, ... }).save();
  // Subcategorías específicas elegidas por el usuario
  for (const subName of subthemes) {

```

```
    await new Subtheme({ name: subName, parent_theme_id: theme._id, ... }).save();
  }
}
```

5.4 Guardar Artículo — Transacción ACID

Al guardar un artículo se usan transacciones de MongoDB para garantizar consistencia entre las tres colecciones involucradas:

```
// metadata-service/server.js — POST /api/v1/articles
const session = await mongoose.startSession();
session.startTransaction();

try {
  // 1. Crear el artículo del usuario
  const newArticle = new Article({ file_hash, title, owner_id,
    theme_id, subtheme_id, size, status: 'available' });
  await newArticle.save({ session });

  // 2. Registrar el nodo primario en StorageMap
  const newStorageMap = new StorageMap({ file_hash, node_id, is_primary: true, status: 'synced'
});
  await newStorageMap.save({ session });

  // 3. Crear tareas de replicación hacia los nodos réplica
  const tasks = replicas.map(replica_id => ({
    file_hash, source_node: node_id, target_node: replica_id, status: 'pending'
  }));
  await ReplicationTask.insertMany(tasks, { session });

  await session.commitTransaction(); // TODO OK → persistir
} catch (error) {
  await session.abortTransaction(); // Cualquier error → rollback
} finally {
  session.endSession();
}
```

5.5 Borrado Distribuido — Reference Counting

```
// metadata-service/server.js — DELETE /api/v1/articles/:hash

// 1. Borrado lógico: elimina el artículo del usuario actual
const deletedArticle = await Article.findOneAndDelete({ file_hash, owner_id }, { session });
```

```
// 2. Conteo de referencias: ¿cuántos usuarios más tienen este archivo?
const remainingOwners = await Article.countDocuments({ file_hash }, { session });

if (remainingOwners > 0) {
  // CASO A: Hay más dueños. El archivo físico se conserva.
  await session.commitTransaction();
  return res.json({ message: 'Borrado lógico. El archivo físico se conserva.' });
}

// CASO B: Eres el último dueño. Se inicia el Hard Delete Distribuido.
const storages = await StorageMap.find({ file_hash }, null, { session });

// Crea una tarea DELETE para cada nodo que tiene el archivo
const deleteTasks = storages.map(s => ({
  file_hash,
  source_node: 'SYSTEM', // Origen especial: el sistema ordena el borrado
  target_node: s.node_id,
  task_type: 'DELETE',
  status: 'pending'
}));
await ReplicationTask.insertMany(deleteTasks, { session });
```

6. Storage Node — services/storage-node/server.js

Cada Storage Node es un servidor gRPC que almacena archivos en disco. Puede haber múltiples instancias corriendo en máquinas distintas. Se registra automáticamente al arrancar y mantiene al sistema informado de su estado vía UDP.

6.1 Registro Automático al Arrancar

```
// storage-node/server.js — registerSelf()
async function registerSelf() {
  try {
    await axios.post(`${METADATA_URL}/nodes/register`, {
      node_id: MY_NODE_ID, // Variable de entorno NODE_ID
      address: MY_ADDRESS // IP:puerto (soporta Tailscale con TAILSCALE_IP)
    });
    console.log(`[${MY_NODE_ID}] Auto-registro exitoso`);
  } catch (error) {
    // Si el metadata-service no está listo, reintenta cada 5 segundos
    setTimeout(registerSelf, 5000);
  }
}
```

6.2 gRPC UploadFile — Recepción de Archivos

```
// storage-node/server.js — uploadFile(call, callback)
function uploadFile(call, callback) {
  let fileStream = null;
  let currentFileHash = "";

  call.on('data', (payload) => {
    if (payload.info) {
      // Primer mensaje: FileInfo
      currentFileHash = payload.info.file_hash;
      const extension = payload.info.extension || '.pdf';
      // Archivo guardado como: uploads/<sha256>.pdf
      const filePath = path.join(UPLOADS_DIR, `${currentFileHash}${extension}`);
      fileStream = fs.createWriteStream(filePath);
    } else if (payload.chunk) {
      // Mensajes siguientes: bytes del archivo
      if (fileStream) fileStream.write(payload.chunk);
    }
  });
}
```

```

    }
  });

  call.on('end', () => {
    if (fileStream) fileStream.end();
    callback(null, {
      file_hash: currentFileHash,
      success: true,
      message: `Guardado en nodo ${process.env.NODE_ID}`
    });
  });
});
}

```

6.3 Worker de Replicación P2P (cada 15 segundos)

El worker corre en background y procesa las tareas pendientes de la base de datos. Soporta dos tipos de tarea: REPLICATE y DELETE:

```

// Cada 15 segundos (con retraso inicial de 5s para esperar el registro)
setInterval(processReplicationTasks, 15000);

async function processReplicationTasks() {
  if (isProcessingTasks) return; // Mutex: evita carrera de tareas
  isProcessingTasks = true;

  // Pide tareas donde este nodo es el source (para REPLICATE)
  // o el target (para DELETE)
  const tasks = await axios.get(
    `${METADATA_URL}/replication-tasks/${MY_NODE_ID}`
  );

  for (const task of tasks.data) {

    if (task.task_type === 'DELETE') {
      // Borra el archivo físicamente del disco
      fs.unlinkSync(path.join(UPLOADS_DIR, taskFile));
      // Notifica al metadata-service que terminó (actualiza StorageMap)
      await axios.post(`${METADATA_URL}/replication-tasks/complete`, {...});
      continue;
    }

    // REPLICATE: busca la IP del nodo destino
    const nodesResponse = await axios.get(`${METADATA_URL}/nodes`);
    const targetNodeInfo = nodesResponse.data.find(n => n.node_id === task.target_node);
  }
}

```

```

if (!targetNodeInfo) continue; // Nodo destino offline, reintenta después

// Abre conexión gRPC al nodo destino y envía el archivo
const client = new storageProto.StorageService(
  targetNodeInfo.address, // Ej: '100.x.x.x:50051'
  grpc.credentials.createInsecure()
);
// Streaming del archivo usando readStream con highWaterMark: 64 KB
}
}

```

6.4 Heartbeat UDP

```

// storage-node/server.js — sendHeartbeat() — cada 5 segundos
function sendHeartbeat() {
  const payload = Buffer.from(JSON.stringify({
    node_id: MY_NODE_ID,
    status: 'up',
    timestamp: Date.now() // El metadata-service usa este valor para last_heartbeat
  }));

  // Envío UDP al metadata-service puerto 3002
  udpClient.send(payload, 0, payload.length,
    METADATA_UDP_PORT, METADATA_HOST, (err) => {
    if (err) console.error('[Heartbeat] Error:', err.message);
  });
}

setInterval(sendHeartbeat, 5000);

```

7. Classifier Service — services/classifier-service/main.py

Es el microservicio de Inteligencia Artificial. Usa un modelo de clasificación zero-shot (sin necesidad de entrenamiento previo) para dos tareas: validar si un PDF es un artículo científico y clasificarlo temáticamente.

7.1 Modelo de IA

```
# main.py — Carga del modelo al arrancar (se hace una sola vez)
from transformers import pipeline

model_name = "MoritzLaurer/mDeBERTa-v3-base-mnli-xnli"
# Características del modelo:
# - Multilingüe (soporta español nativamente)
# - Arquitectura DeBERTa v3 — 278M parámetros
# - Entrenado en MNLI + XNLI para zero-shot classification
# - Peso aproximado: ~1.5 GB (se descarga en el primer arranque)

# Usa GPU automáticamente si está disponible (CUDA)
device = 0 if torch.cuda.is_available() else -1

classifier = pipeline("zero-shot-classification",
                      model=model_name,
                      device=device)
```

7.2 Endpoint POST /classify

```
# main.py — Endpoint de clasificación
class ClassificationRequest(BaseModel):
    text: str # Texto extraído del PDF (máx 2500 chars)
    candidate_labels: List[str] # Etiquetas candidatas para la clasificación

@app.post("/classify")
async def classify_text(request: ClassificationRequest):
    result = classifier(
        request.text,
        request.candidate_labels,
        multi_label=False # Solo una etiqueta gana
    )
    # Resultado ordenado por puntaje de mayor a menor
    return {
```

```

    "best_label": result['labels'][0], # Categoría ganadora
    "confidence": result['scores'][0], # Confianza (0.0 - 1.0)
    "all_scores": dict(zip(result['labels'], result['scores']))
  }

@app.get("/health")
async def health():
  return { "status": "AI Model is ready",
          "device": "gpu" if device == 0 else "cpu" }

```

7.3 Lógica de Validación en el Gateway

El classifier-service se llama dos veces por cada archivo subido. La primera llamada valida la estructura del documento, la segunda lo clasifica:

```

// api-gateway/server.js — Llamada 1: Validación IMRyD
const validationLabels = [
  'artículo científico (IMRyD)',
  'documento genérico',
  'publicidad o spam'
];

const validationResponse = await axios.post(
  'http://classifier-service:8000/classify',
  { text: extractedText, candidate_labels: validationLabels }
);

// Umbral: confidence >= 0.4 para considerar artículo científico válido
const isValid = validationResponse.data.best_label === 'artículo científico (IMRyD)'
  && validationResponse.data.confidence >= 0.4;

if (!isValid) {
  fs.unlinkSync(tempPath);
  return res.status(400).json({
    error: 'El archivo no cumple la estructura IMRyD.'
  });
}

// api-gateway/server.js — Llamada 2: Clasificación temática
// candidateLabels = nombres de temas y subtemas del usuario
const aiResponse = await axios.post(
  'http://classifier-service:8000/classify',
  { text: extractedText, candidate_labels: candidateLabels }
);

```

```
// Umbral: confidence > 0.3 para asignar la categoría ganadora
if (aiResponse.data.confidence > 0.3) {
  finalThemeId = categoryMap[aiResponse.data.best_label].theme_id;
  finalSubthemeId = categoryMap[aiResponse.data.best_label].subtheme_id;
}
// Si confidence <= 0.3 → se asigna a la categoría 'General' por defecto
```

8. Frontend — Frontend/scidist-frontend/

El frontend es una Single Page Application (SPA) construida con React 19 y Vite 8. Tiene cuatro rutas principales: landing, autenticación, dashboard de usuario y panel de administrador.

8.1 Configuración de Axios — axiosConfig.js

```
// src/api/axiosConfig.js
import axios from 'axios';

// IP de Tailscale de la máquina donde corre el API Gateway
// CAMBIAR a tu IP antes de ejecutar
const GATEWAY_URL = "http://100.119.151.81:3000/api/v1";

const api = axios.create({
  baseURL: GATEWAY_URL,
  headers: { 'Content-Type': 'application/json' }
});

export default api;

// Uso en cualquier componente:
// import api from '../api/axiosConfig';
// await api.get('/files/user/123');
// await api.post('/upload', formData, { headers: { 'x-user-id': id } });
```

8.2 Rutas — AppRouter.jsx

```
// src/routes/AppRouter.jsx
function AppRouter() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Scidist />} />
        <Route path="/auth" element={<AuthPage />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/dashboard-admi" element={<AdminDashboard />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```
// Navegación real entre rutas:
// Scidist.jsx → navigate('/auth') al hacer clic en 'Ingresar'
// AuthPage.jsx → navigate('/dashboard') login exitoso usuario normal
// AuthPage.jsx → navigate('/dashboard-admin') login exitoso admin
// AppLayout.jsx → navigate('/') al hacer clic en 'Salir'
```

8.3 AuthPage.jsx — Login y Registro

Gestiona dos flujos: login y registro. Carga el catálogo de categorías del backend al montar y construye el payload de preferencias con los checkboxes:

```
// src/pages/AuthPage.jsx — Estado del componente
const [activeTab, setActiveTab] = useState('login');
const [loginData, setLoginData] = useState({ email: "", password: "" });
const [regData, setRegData] = useState({ firstName, lastName, email, password,
confirmPassword });
const [selectedTopics, setSelectedTopics] = useState({});
// Ejemplo: { 'Redes': ['Protocolos', 'TCP'], 'Linux': [] }

// Carga el catálogo al montar el componente
useEffect(() => {
  api.get('/categories/available').then(r => setAvailableCategories(r.data));
}, []);

// Payload de registro que se envía al backend
const payload = {
  username: `${firstName} ${lastName}`,
  email: regData.email,
  password: regData.password,
  preferences: selectedTopics // { 'Redes': ['Protocolos'] }
};
await api.post('/auth/register', payload);

// Después de login exitoso, guarda en localStorage:
localStorage.setItem('user', JSON.stringify(response.data.user));
// response.data.user = { id, username, email, is_admin }
```

8.4 Dashboard.jsx — Página Principal del Usuario

Componente central. Carga los archivos del usuario, gestiona la subida con validación IA, permite ver/descargar/eliminar archivos y gestionar categorías personales:

```
// src/pages/Dashboard.jsx — Estados principales
const [files, setFiles] = useState([]);
```

```

const [isLoadingFiles, setLoading] = useState(true);
const [isUploading, setIsUploading] = useState(false);
const [currentFilter, setFilter] = useState('todos');
const [refreshKey, setRefreshKey] = useState(0); // Fuerza re-fetch
const [showModal, setShowModal] = useState(false); // Visor de archivos
const [viewingUrl, setViewingUrl] = useState(null); // Blob URL del PDF
const [showCatModal, setShowCatModal] = useState(false); // Gestor categorías

// Carga de archivos al montar y cuando refreshKey cambia
useEffect(() => {
  const storedUser = JSON.parse(localStorage.getItem('user'));
  api.get(`/files/user/${storedUser.id}`).then(r => setFiles(r.data));
}, [refreshKey]);

// Estadísticas derivadas con useMemo (no re-calculan en cada render)
const stats = useMemo(() => ({
  total: files.length,
  areas: new Set(files.map(f => f.category)).size,
  size: formatBytes(files.reduce((a, f) => a + f.size, 0)),
  types: new Set(files.map(f => f.name.split('.').pop())).size
}), [files]);

// Archivos filtrados por categoría activa
const filteredFiles = useMemo(() =>
  currentFilter === 'todos'
    ? files
    : files.filter(f =>
        f.category === currentFilter || f.subcategory === currentFilter
      ),
  [currentFilter, files]);

```

8.5 handleFiles() — Subida con IA

```

// src/pages/Dashboard.jsx — handleFiles()
const handleFiles = async (selectedFiles) => {
  const storedUser = JSON.parse(localStorage.getItem('user'));
  setIsUploading(true);

  for (const file of Array.from(selectedFiles)) {
    const formData = new FormData();
    formData.append('file', file);

    try {

```

```

    await api.post('/upload', formData, {
      headers: {
        'Content-Type': 'multipart/form-data',
        'x-user-id': storedUser.id // Identificador del propietario
      }
    });
  } catch (error) {
    // Muestra el error exacto del backend (ej: 'No cumple estructura IMRyD')
    alert('Fallo al subir ${file.name}: ${error.response?.data?.error}');
  }
}

setIsUploading(false);
setRefreshKey(prev => prev + 1); // Re-carga la galería automáticamente
};

```

8.6 handleViewFile() — Visor de Archivos

```

// src/pages/Dashboard.jsx — handleViewFile(file)
const handleViewFile = async (file) => {
  setActiveFileTitle(file.name);
  setIsViewLoading(true);
  setShowModal(true);

  // Descarga el archivo como Blob binario
  const response = await api.get(`/download/${file.hash}`, {
    responseType: 'blob',
    headers: { 'x-user-id': storedUser.id }
  });

  if (file.name.endsWith('.pdf')) {
    // Crea URL temporal del Blob con tipo explícito application/pdf
    const pdfBlob = new Blob([response.data], { type: 'application/pdf' });
    setViewingUrl(URL.createObjectURL(pdfBlob)); // ej: blob:http://localhost/...
    setFileType('pdf');
    // Se usa <embed src={url}> en el Modal para renderizar el PDF
  } else {
    // Para texto: usa FileReader para leer el contenido
    const reader = new FileReader();
    reader.onload = (e) => { setViewingUrl(e.target.result); setFileType('text'); };
    reader.readAsText(response.data);
    // Se usa <pre> con fuente monoespaciada para mostrar el texto
  }
}

```

```

};

// handleCloseModal libera la memoria del Blob al cerrar
const handleCloseModal = () => {
  setShowModal(false);
  URL.revokeObjectURL(viewingUrl); // Importante: evita memory leak
  setViewingUrl(null);
};

```

8.7 Gestión de Categorías — Modal Organizar Carpetas

```

// src/pages/Dashboard.jsx — fetchCategoryData()
// Carga en paralelo el árbol actual del usuario y el catálogo global
const [userTreeRes, catalogRes] = await Promise.all([
  api.get('/themes/me', { headers: { 'x-user-id': storedUser.id } }),
  api.get('/categories/catalog')
]);
setCategoryTree(userTreeRes.data); // Árbol actual del usuario
setGlobalCatalog(catalogRes.data.filter(c => c.name !== 'general'));

// handleAddCategory(): Construye el payload de preferencias
const payload = {
  preferences: {
    [selectedCatalogTheme]: selectedCatalogSubtheme ? [selectedCatalogSubtheme] : []
  }
};
await api.post('/themes/me', payload, { headers: { 'x-user-id': storedUser.id } });

// handleDeleteCategory(): Elimina tema o subtema
const endpoint = isSubtheme
  ? `/subthemes/me/${id}`
  : `/themes/me/${id}`;
await api.delete(endpoint, { headers: { 'x-user-id': storedUser.id } });
// El backend reasigna automáticamente los artículos huérfanos a General/Otros

```

8.8 AppLayout.jsx — Navbar + Sidebar Dinámico

El Layout es el esqueleto que envuelve al Dashboard. Carga las categorías del usuario desde el backend y construye el sidebar con acordeones para los subtemas:

```

// src/components/layout/AppLayout.jsx
// Props recibidos del Dashboard:
// - children: el contenido de la página
// - onSelectCategory: callback para cambiar el filtro activo

```

```
// - categoryCounts: { todos: 5, Redes: 3, Protocolos: 2, ... }
// - refreshKey: cuando cambia, vuelve a cargar las categorías

// Carga las categorías del usuario al montar
useEffect(() => {
  const user = JSON.parse(localStorage.getItem('user'));
  if (!user) { navigate('/'); return; } // Guard de autenticación
  api.get(`/themes/user/${user.id}`).then(res => {
    const categoriesWithIcons = res.data.map(cat => ({
      ...cat,
      icon: getIconForCategory(cat.name) // Ícono según el nombre del tema
    }));
    setUserCategories(categoriesWithIcons);
  });
}, [navigate, refreshKey]);

// Sidebar acordeón: cada tema se expande para mostrar sus subtemas
const toggleAccordion = (id) => {
  setOpenAccordions(prev => ({ ...prev, [id]: !prev[id] }));
};

// Logout: limpia localStorage y redirige a la raíz
const handleLogout = () => {
  localStorage.removeItem('user');
  navigate('/');
};
```

8.9 Helpers — src/utils/helpers.js

```
// Íconos para las categorías del sidebar
export const getIconForCategory = (name) => {
  const lowerName = name?.toLowerCase() || '';
  if (lowerName.includes('ia') || lowerName.includes('inteligencia')) return 'bi-robot';
  if (lowerName.includes('redes')) return 'bi-hdd-network';
  if (lowerName.includes('dev') || lowerName.includes('desarrollo')) return 'bi-code-slash';
  if (lowerName.includes('linux')) return 'bi-ubuntu';
  if (lowerName.includes('general')) return 'bi-folder-symlink';
  return 'bi-folder2'; // Ícono por defecto
};

// Íconos por extensión de archivo
export const getFileIcon = (filename) => {
  const ext = filename.split('.').pop().toLowerCase();
```

```

const icons = {
  pdf: 'bi-filetype-pdf text-danger',
  doc: 'bi-filetype-docx text-primary', docx: 'bi-filetype-docx text-primary',
  js: 'bi-filetype-js text-warning', py: 'bi-filetype-py text-info',
  jpg: 'bi-image text-success', png: 'bi-image text-success',
  txt: 'bi-filetype-txt text-secondary', zip: 'bi-file-zip text-muted'
};
return icons[ext] || 'bi-file-earmark text-muted';
};

// Formato de bytes (2 decimales por defecto)
export const formatBytes = (bytes, decimals = 2) => {
  if (!+bytes) return '0 Bytes';
  const k = 1024;
  const sizes = ['Bytes', 'KB', 'MB', 'GB', 'TB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return `${parseFloat((bytes / Math.pow(k, i)).toFixed(decimals))} ${sizes[i]}`;
};

// Formato de fecha ISO → '04 may. 2026' (locale es-MX)
export const formatDate = (dateString) => {
  if (!dateString) return 'Fecha desconocida';
  return new Date(dateString).toLocaleDateString('es-MX',
    { day: '2-digit', month: 'short', year: 'numeric' });
};

```

9. Dashboard-admi.jsx — Panel de Administrador

El panel de administrador es accesible únicamente para usuarios con `is_admin: true` en la base de datos. Permite monitorear nodos, gestionar usuarios y ver el inventario global del sistema.

9.1 Secciones del Panel Admin

Sección	Datos que muestra y endpoints consultados
Vista General	Resumen del sistema. KPIs de nodos, usuarios, artículos y tareas de replicación
Nodos de Almacenamiento	Estado online/offline de cada nodo, heartbeat, IP, health. GET <code>/admin/nodes</code>
Gestión de Usuarios	Lista, crea, edita y elimina usuarios. GET/POST/PUT/DELETE <code>/admin/users/:id</code>
Registros del Sistema	Últimas 100 tareas de replicación con status y timestamps. GET <code>/admin/replications</code>
Configuración	Árbol de categorías de cada usuario. GET <code>/admin/users/:id/themes</code>

9.2 Header x-user-id en cada petición admin

```
// Dashboard-admi.jsx — Helper para headers de autenticación
function adminHeaders() {
  const stored = localStorage.getItem('user');
  if (!stored) return {};
  return { 'x-user-id': JSON.parse(stored).id };
}

// Uso en cada llamada al API:
await api.get('/admin/users', { headers: adminHeaders() });
await api.delete('/admin/users/${id}', { headers: adminHeaders() });
// El checkAdmin middleware en el gateway verifica que este ID tenga is_admin: true
```

9.3 StatusBadge — Componente de Estado

```
// Dashboard-admi.jsx — Mapa de estilos por estado
const STATUS_META = {
  done: { bg: 'rgba(34,197,94,0.15)', color: '#22c55e', label: 'Completado' },
  pending: { bg: 'rgba(234,179,8,0.15)', color: '#eab308', label: 'Pendiente' },
}
```

```
in_progress: { bg: 'rgba(59,130,246,0.15)', color: '#3b82f6', label: 'En proceso' },
failed:      { bg: 'rgba(239,68,68,0.15)', color: '#ef4444', label: 'Fallido' },
up:          { bg: 'rgba(34,197,94,0.15)', color: '#22c55e', label: 'Online' },
down:        { bg: 'rgba(239,68,68,0.15)', color: '#ef4444', label: 'Offline' },
synced:      { bg: 'rgba(34,197,94,0.15)', color: '#22c55e', label: 'Sincronizado' },
// ... más estados
};

// Componente: renderiza un badge con color según el status
function StatusBadge({ status }) {
  const s = STATUS_META[status] || { bg: 'rgba(156,163,175,0.15)',
    color: '#9ca3af', label: status };
  return <span style={{ background: s.bg, color: s.color, ... }}>{s.label}</span>;
}
```

10. Dockerfiles — Contenedores de Producción

10.1 api-gateway/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
RUN mkdir -p temp_uploads # Carpeta temporal para multer
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

10.2 metadata-service/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . . # Incluye server.js, models/index.js y el .env
EXPOSE 3001 # HTTP
# UDP 3002 debe mapearse también en docker-compose
CMD ["node", "server.js"]
```

10.3 storage-node/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
# IMPORTANTE: la carpeta proto/ y uploads/ se montan desde docker-compose
EXPOSE 50051 # Puerto gRPC
CMD ["node", "server.js"]

# Variables de entorno necesarias al correr el contenedor:
# NODE_ID=storage-node-1
# PORT=50051
```

```
# METADATA_HOST=metadata-service
# TAILSCALE_IP=100.x.x.x (si se usa Tailscale VPN)
```

10.4 classifier-service/Dockerfile

```
FROM python:3.9-slim
WORKDIR /app

# Instala dependencias de compilación para torch
RUN apt-get update && apt-get install -y build-essential

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Primera ejecución: descarga el modelo mDeBERTa (~1.5 GB desde HuggingFace)

COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

⚠ El classifier-service descarga el modelo mDeBERTa (~1.5 GB) en el primer arranque desde HuggingFace Hub. Para evitar la descarga en cada recreación del contenedor, monta un volumen Docker en la carpeta de caché de HuggingFace (~/.cache/huggingface).

10.5 Ejemplo de docker-compose.yml

```
# docker-compose.yml (referencia — no incluido en el repo aún)
version: '3.8'
services:

  api-gateway:
    build: ./services/api-gateway
    ports: ['3000:3000']
    depends_on: [metadata-service, classifier-service]

  metadata-service:
    build: ./services/metadata-service
    ports: ['3001:3001', '3002:3002/udp']
    environment:
      MONGO_URI: mongodb://mongo:27017/scidist?replicaSet=rs0

  storage-node-1:
    build: ./services/storage-node
    environment:
```

```
NODE_ID: storage-node-1
PORT: '50051'
METADATA_HOST: metadata-service
volumes:
  - ./proto:/app/proto
  - storage1_data:/app/uploads

classifier-service:
  build: ./services/classifier-service
  ports: ['8000:8000']
  volumes:
    - hf_cache:/root/.cache/huggingface # Cachea el modelo

volumes:
  storage1_data:
  hf_cache:
```

11. Referencia Rápida

11.1 Puertos y Servicios

Servicio	Puerto / Protocolo / Tecnología
api-gateway	:3000 — HTTP/REST — Node.js Express
metadata-service	:3001 — HTTP/REST — Node.js Express + Mongoose
metadata-service	:3002 — UDP (solo escucha heartbeats) — dgram
storage-node	:50051 — gRPC streaming — Node.js @grpc/grpc-js
classifier-service	:8000 — HTTP/REST — Python FastAPI + uvicorn
scidist-frontend	:5173 (dev) — HTTP — Vite dev server

11.2 Variables de Entorno

Variable	Servicio y descripción
MONGO_URI	metadata-service — URI completa de MongoDB con Replica Set
NODE_ID	storage-node — Identificador único del nodo (ej: 'storage-node-1')
PORT	storage-node — Puerto gRPC (default: 50051)
METADATA_HOST	storage-node — Hostname del metadata-service (default: 'metadata-service')
TAILSCALE_IP	storage-node — IP de Tailscale VPN para registrar la dirección pública del nodo
GATEWAY_URL	Frontend (axiosConfig.js) — URL del API Gateway desde el navegador

11.3 Funciones y Hooks del Frontend

Función / Hook	Archivo y descripción
handleLoginSubmit()	AuthPage.jsx — POST /auth/login, guarda user en localStorage, redirige
handleRegisterSubmit()	AuthPage.jsx — POST /auth/register con preferences, crea árbol de carpetas

handleToggleSubtheme()	AuthPage.jsx — Agrega/quita subtemas del objeto selectedTopics
handleFiles()	Dashboard.jsx — Sube PDFs uno a uno con x-user-id header
handleViewFile()	Dashboard.jsx — Descarga Blob y crea URL temporal para el visor
handleDownloadFile()	Dashboard.jsx — Descarga forzada con <a download>
handleDeleteFile()	Dashboard.jsx — DELETE /delete/:hash con confirmación
handleOpenCategories()	Dashboard.jsx — Carga árbol + catálogo en paralelo, abre modal
handleAddCategory()	Dashboard.jsx — POST /themes/me con la selección del catálogo
handleDeleteCategory()	Dashboard.jsx — DELETE /themes/me/:id o /subthemes/me/:id
fetchCategoryData()	Dashboard.jsx — Promise.all([/themes/me, /categories/catalog])
fetchCategories()	AppLayout.jsx — GET /themes/user/:id, inyecta íconos
handleCategoryClick()	AppLayout.jsx — Cierra sidebar, llama onSelectCategory(name)
toggleAccordion()	AppLayout.jsx — Abre/cierra subcategorías en el sidebar
handleLogout()	AppLayout.jsx — localStorage.removeItem('user'), navigate('/')
getIconForCategory()	helpers.js — Retorna clase Bootstrap Icon según el nombre del tema
getFileIcon()	helpers.js — Retorna clase Bootstrap Icon según la extensión
formatBytes()	helpers.js — Convierte bytes a KB/MB/GB con n decimales
formatDate()	helpers.js — Fecha ISO → '04 may. 2026' (es-MX)
adminHeaders()	Dashboard-admi.jsx — { 'x-user-id': userId } del localStorage
timeAgo()	Dashboard-admi.jsx — Diferencia de tiempo en s/m/h/d
hashShort()	Dashboard-admi.jsx — Trunca el hash: '1a2b...ef3d'

Scidist-Archive

Manual de Usuario — Referencia Técnica Completa
github.com/alexvera23/Scidist-archive · © 2026 Scidist-Archive